



LV05 재귀 함수에 대한 사실과 오해

https://youtu.be/17_ybrN6Bss?feature=shared

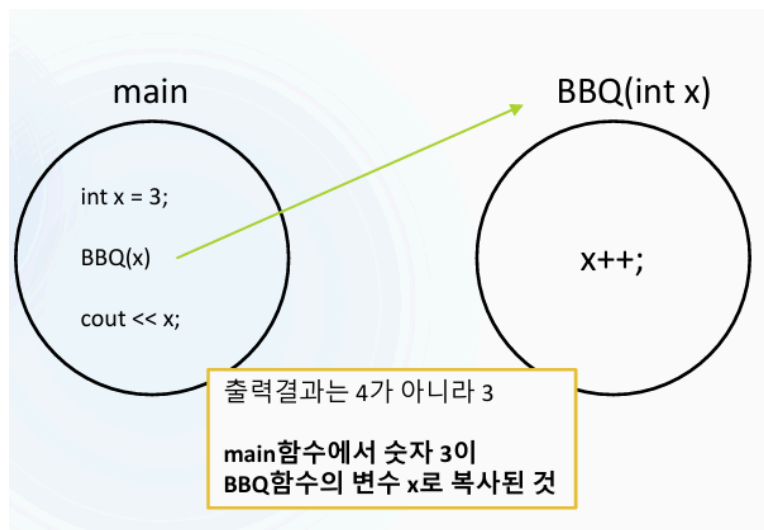
C++ 함수 호출과 재귀함수

함수 복습하기

함수 호출의 기본 원리

함수가 호출될 때마다 새로운 ****스택 프레임(Stack Frame)****이 생성됩니다. 각 함수는 독립적인 메모리 공간을 가지며, 매개변수와 지역변수들이 이 공간에 저장됩니다.

함수 호출 과정 시각화



함수 호출 메커니즘:

1. **매개변수 복사:** main의 x값(3)이 BBQ 함수의 매개변수 x로 복사
2. **독립적 실행:** BBQ 함수 내부에서 x++은 복사된 값만 변경
3. **원본 보존:** main 함수의 x는 여전히 3

4. 함수 종료: BBQ 함수 종료 후 main으로 복귀

return 키워드의 중요성

return 키워드를 통해 나를 호출하기 전에 함수로 실행 프로세스를 넘겨준다는 의미였다.

함수마다 공간이 다른 공간이기 때문에 같은 지역변수 이름이 허용된다.

```
#include <iostream>
using namespace std;

void BBQ(int x)
{
    cout << "BBQ 함수 내부 x: " << x << endl; // 3
    x++;
    cout << "BBQ 함수 x++ 후: " << x << endl; // 4
}

int main()
{
    int x = 3;
    cout << "main 함수 x: " << x << endl; // 3

    BBQ(x);

    cout << "BBQ 호출 후 main x: " << x << endl; // 여전히 3

    return 0;
}
```

재귀 함수 (Recursive Function)

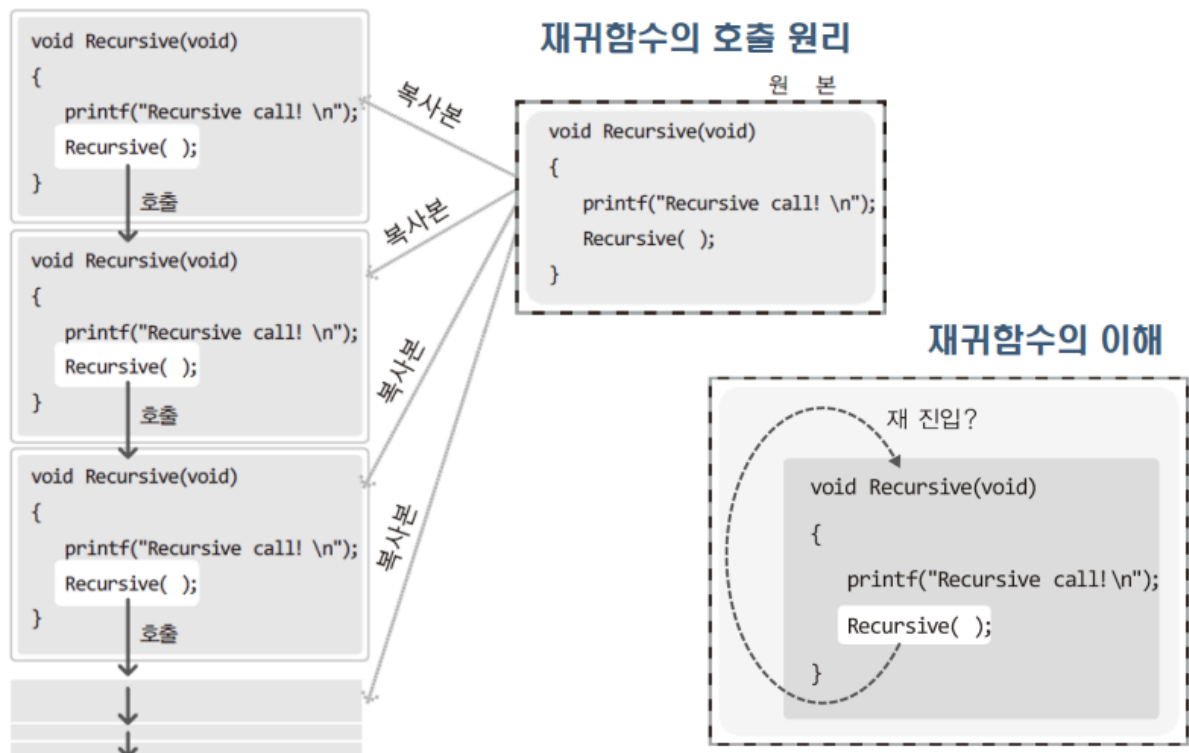
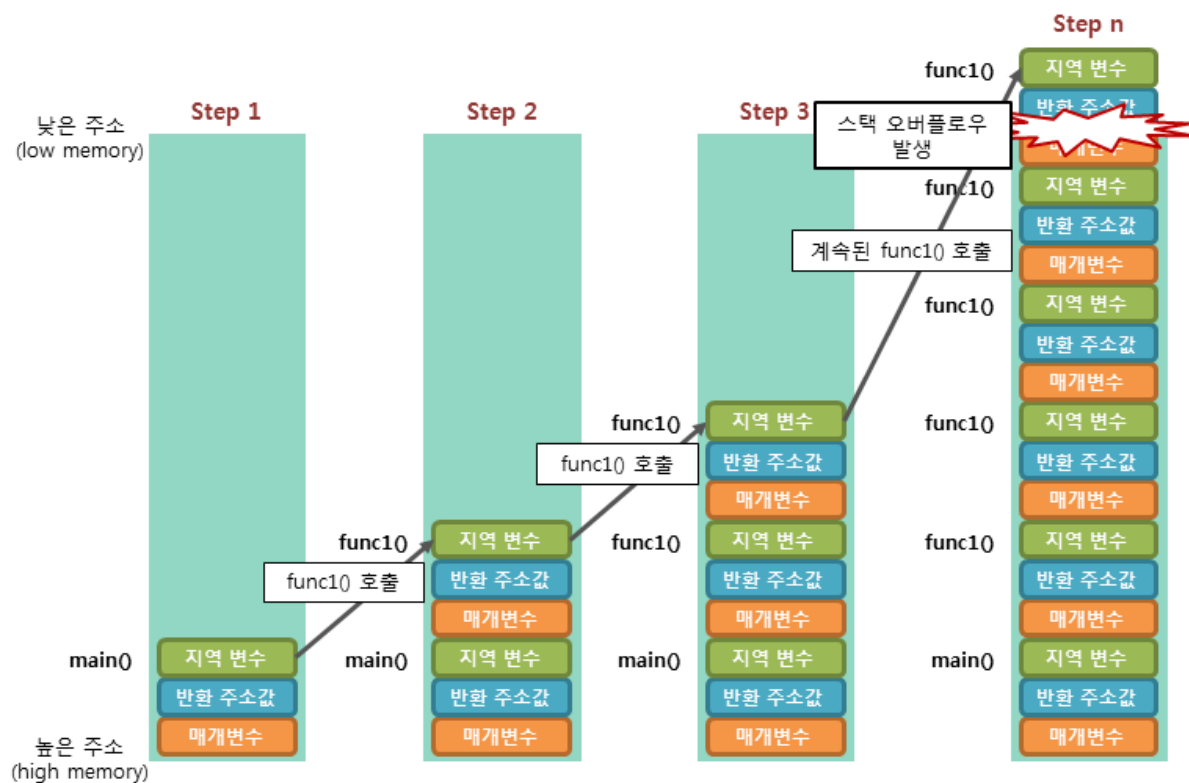
재귀 함수의 정의

대부분의 인터넷 설명을 보다보면 재귀함수는 자기 자신을 호출하는 것이라고 표현한다.

하지만 실상은 반은 맞고 반은 틀린 말이다. 해당 표현으로 이해를 하게되면 결국 같은 함수로 오해하여 이해 함수 없기 때문이다.

재귀함수란 흔히 나 자신을 호출하는 함수라고 많이들 이야기 하지만 실제로는 코드가 재활용되어 같은 이름의(다른함수)를 또 호출하는 것이다.

결론적으로 코드만 같고 다른 함수로 이해 하면 구조를 파악하기 쉽다.



윤성우의 열혈 자료구조

재귀 함수의 메모리 구조

재귀 함수는 ****스택(Stack)****이라는 메모리 구조를 사용합니다:

장점:

- 변수를 여러 만들 필요가 없다.
- 예들들어 현재 상태를 저장해야 할 경우 tmp 변수를 만들기보다 상태를 메서드로 재귀적으로 호출하면서 변경된 상태를 전달 함으로써 변수의 수를 줄일 수 있습니다

while문이나 for문같은 반복문을 사용하지 않아도 되기에 코드가 간결해집니다.

단점:

- 지속적으로 함수를 호출하게 되면서 지역변수, 매개변수, 반환값을 모두 process stack에 저장해야합니다
- 그리고 이런 과정은 선언한 변수의 값만 사용하는 반복문에 비해서 메모리를 더 많이 사용하게 되고, 이는 속도 저하로 이어집니다

함수 호출 -> 복귀를 위한 컨텍스트 스위칭에 비용이 발생하게 됩니다.

기본적인 재귀 함수 예제

```
#include <iostream>
using namespace std;

// ABC → test → main 순서로 배워야한다
// 가장 나중에 들어온 데이터가 가장 먼저 빠져나가는 것 = stack

// 재귀함수를 통한 나 자신을 호출하는 함수라고 많이들 하지만
// 실제로는 코드가 재활용되어 똑같은 이름의 다른 함수를 호출하는 것이다.
// 5. main → test → test → test .....
// 재귀에서 스택이 쌓이는 것이다.
// 아래의 코드를 r5 하면 'Stack Overflow' 표시가 나온다

void test(int x)
{
    // 무한정 호출되는 함수를 방지해야 하는 게 급선무이다.
    // test(x);

    // 아래의 함수를 호출함 4025번까지 호출하고나서 stack Overflow 된다
    // test(x + 1);
```

```

    if (x == 3)
    {
        return;
        // test(2) 로 돌아감 -> test(1) → ... main 으로 돌아감
    }

    test(x + 1);
    cout << x << endl;
}

int main()
{
    test(0);

    return 0;
}

```

재귀 함수 실행 과정 상세 분석

스택 프레임 분석:

호출 순서: main(0) → test(0) → test(1) → test(2) → test(3)
 반환 순서: test(3) return → test(2) → test(1) → test(0) → main

실행 흐름:

1. `test(0)` 호출: $x=0$, $x \neq 3$ 이므로 `test(1)` 호출
2. `test(1)` 호출: $x=1$, $x \neq 3$ 이므로 `test(2)` 호출
3. `test(2)` 호출: $x=2$, $x \neq 3$ 이므로 `test(3)` 호출
4. `test(3)` 호출: $x=3$, $x == 3$ 이므로 `return` 실행
5. `test(2)` 로 복귀: `cout << 2` 실행 후 종료
6. `test(1)` 로 복귀: `cout << 1` 실행 후 종료
7. `test(0)` 로 복귀: `cout << 0` 실행 후 종료

출력 결과: 2 1 0

배열을 이용한 재귀 함수

```
#include <iostream>
using namespace std;

int arr[5] = { 5, 7, 1, 2, 3 };

void test(int x)
{
    if (x == 5)
    {
        return;
    }

    cout << arr[x] << " ";
    test(x + 1);
    cout << arr[x] << " ";
}

int main()
{
    test(0);

    return 0;
}
```

실행 과정:

1. `test(0)` : arr[0]=5 출력, `test(1)` 호출
2. `test(1)` : arr[1]=7 출력, `test(2)` 호출
3. `test(2)` : arr[2]=1 출력, `test(3)` 호출
4. `test(3)` : arr[3]=2 출력, `test(4)` 호출
5. `test(4)` : arr[4]=3 출력, `test(5)` 호출
6. `test(5)` : return 실행
7. `test(4)` 로 복귀: arr[4]=3 출력
8. `test(3)` 로 복귀: arr[3]=2 출력

9. `test(2)` 로 복귀: `arr[2]=1` 출력
10. `test(1)` 로 복귀: `arr[1]=7` 출력
11. `test(0)` 로 복귀: `arr[0]=5` 출력

출력 결과: `5 7 1 2 3 3 2 1 7 5`

재귀함수를 이용한 배열 순회

```
#include <iostream>
using namespace std;

// 문제
// 출력결과 : 0 1 2 3
int arr[5] = { 5, 7, 1, 2, 3 };

void test(int x)
{
    // 무한정 호출되는 함수를
    // 방지해야 하는 게 급선무이다.
    if (x == 3)
    {
        return;
    }

    cout << arr[x] << endl;
    test(x + 1);
}

int main()
{
    test(0);

    return 0;
}
```

순방향 출력 결과: `5 7 1` (`arr[0]`, `arr[1]`, `arr[2]`)

재귀 함수의 다양한 활용

팩토리얼 계산

```
int factorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }

    return n * factorial(n - 1);
}

// factorial(5) = 5 * factorial(4) = 5 * 4 * 3 * 2 * 1 = 120
```

스택 오버플로우 방지

스택이 넘어가면 프로그램이 강제 종료 될수 있기에 예외처리를 넣어주는 것이 실무에서는 좋다.

```
void safeRecursion(int x, int maxDepth = 1000)
{
    if (x >= maxDepth) // 최대 깊이 제한
    {
        cout << "Maximum recursion depth reached!" << endl;
        return;
    }

    // 재귀 로직
    safeRecursion(x + 1, maxDepth);
}
```

꼬리 재귀 최적화(중요사항 X 읽고 이렇게 있구나 하고 넘어가세요.)

▼ *꼬리 재귀(Tail Recursion)*



재귀 호출이 함수의 "가장 마지막에" 실행되고, 그 결과를 그대로 반환 (return)하는 재귀 함수를 의미한다.

즉,

- 재귀 호출 이후에 추가 작업이 없고,
- 바로 **return 함수호출**로 끝나는 형태야.



일반 재귀 vs 꼬리 재귀 예제



일반 재귀 (꼬리 아님)

```
int factorial(int n)
{
    if (n <= 1) return 1;
    return n * factorial(n - 1); // ← 호출 뒤에 곱셈 연산이 남아 있음
}
```



꼬리 재귀

```
int factorial_tail(int n, int acc = 1)
{
    if (n <= 1) return acc;
    return factorial_tail(n - 1, n * acc); // ← 재귀 호출이 마지막 동작
}
```



꼬리 재귀의 장점

- 스택 프레임을 줄일 수 있음 (최적화 가능)
 - 대부분의 언어/컴파일러가 ****Tail Call Optimization (TCO)****을 지원할 경우
 - 꼬리 재귀는 마치 반복문처럼 처리됨 → **메모리 낭비 없음**
- 깊은 재귀도 스택 오버플로우 없이 실행 가능



꼬리 재귀 최적화(Tail Call Optimization, TCO)

언어	꼬리 재귀 최적화 지원 여부
C/C++	일부 컴파일러만 지원 (옵션 필요)
Java	❌ 미지원 (JVM 구조상 불리함)
Python	❌ 미지원
Scheme/Lisp	✅ 완전 지원
Rust	❌ 기본적으로 미지원
Swift	✅ 일부 지원 (최적화됨)
Haskell	✅ 강력하게 지원

반복문과 꼬리 재귀는 구조상 거의 같음

꼬리 재귀는 다음처럼 반복문으로 쉽게 바꿀 수 있어:

```
int factorial_loop(int n)
{
    int acc = 1;
    while (n > 1)
    {
        acc *= n;
        n--;
    }
    return acc;
}
```

✨ 정리

항목	내용
의미	함수가 자기 자신을 호출하되, 마지막 동작 으로만 호출
장점	스택 절약 , 최적화 가능
특징	재귀 호출 뒤에 추가 연산이 없어야 함
예시	<code>return f(n-1);</code> ← 꼬리 재귀 / <code>return n * f(n-1);</code> ← 일반 재귀

필요하다면 너가 사용 중인 언어나 문제 상황에 맞는 꼬리 재귀 예제도 만들어줄게.

C++이나 Unity C#에서도 적용 가능해. 원해?

```
// 일반 재귀 (비효율적)
int factorial(int n)
{
    if (n <= 1) return 1;
    return n * factorial(n - 1); // 곱셈 후 반환
}

// 꼬리 재귀 (효율적)
int factorialTail(int n, int result = 1)
{
    if (n <= 1) return result;
    return factorialTail(n - 1, n * result); // 즉시 재귀 호출
}
```

재귀 vs 반복문 비교

성능 비교

특성	재귀 함수	반복문
메모리 사용	많음 (스택 프레임)	적음 (지역변수만)
속도	느림 (함수 호출 오버헤드)	빠름
코드 가독성	높음 (직관적)	보통
디버깅	어려움	쉬움
스택 오버플로우	위험	안전

사용 권장 사항

재귀 함수를 사용하기 좋은 경우:

- 트리나 그래프 탐색
- 분할 정복 알고리즘
- 수학적 점화식 구현
- 코드 가독성이 중요한 경우

반복문을 사용하기 좋은 경우:

- 단순 반복 작업

- 성능이 중요한 경우
- 메모리 사용량을 줄여야 하는 경우
- 깊은 재귀가 예상되는 경우

재귀 함수는 복잡한 문제를 간단하게 표현할 수 있는 강력한 도구이지만, 성능과 메모리 사용량을 고려하여 적절히 사용해야 합니다. 특히 스택 오버플로우를 방지하기 위한 종료 조건 설정이 매우 중요합니다.

"강의는 많은데, 내 실력은 왜 그대로일까?"

혼자서 공부하다 보면

이런 생각 들지 않으셨나요?

- 강의는 다 듣고도 직접 코드는 못 짜겠고,
- 복습할 땐 어디서부터 다시 시작해야 할지 막막하고,
- 질문하려 해도 물어볼 사람이 없고,
- 유튜브 영상도 정답만 보고 따라 치는 느낌...

그렇다면 지금이 바로

"나만을 위한 코칭"이 필요한 순간입니다.

당신도 할 수 있습니다.

지금 멤버십을 넘어, 코칭에 도전해보세요.

수많은 수강생들이 얀얀코딩 코칭으로 넥슨, 크래프톤, NC 등 입사에 성공했습니다.

 프리미엄 코칭 안내 바로가기

 또는 카톡 오픈채팅: 얀얀코딩 상담방

지금도 코딩을 '따라 치기만' 하고 계신가요?

이젠 혼자 설계하고, 스스로 코딩하는 법을 배워야 할 때입니다.

얀얀코딩이 옆에서 함께하겠습니다. 💪

▼ oopy:hide

연습문제

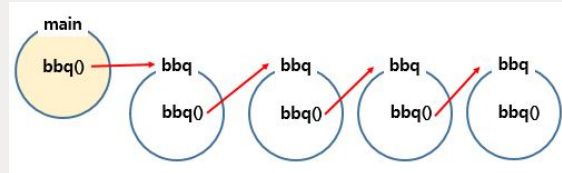


무한 재귀 막기

문제 1번

아래 그림과 같이 재귀 함수를 구현해주세요.

(전역변수를 쓰지 않습니다)



입출력 값이 없는 문제입니다.

- -----

Trace 연습을 많이 해야합니다.

F10, F11, ctrl + F10 버튼을 이용해서

능숙해지도록 연습을 꼭 해보세요!!

Level20 번지점프 [난이도 : 3]

문제 2번

숫자 n을 입력 받으세요.

숫자 n부터 0까지 Count down 했다가
다시 돌아오는 수를 출력 하시면 됩니다.

ex) 4

4 3 2 1 0 1 2 3 4

ex) 6

6 5 4 3 2 1 0 1 2 3 4 5 6

입력 예제

4

출력 결과

4 3 2 1 0 1 2 3 4

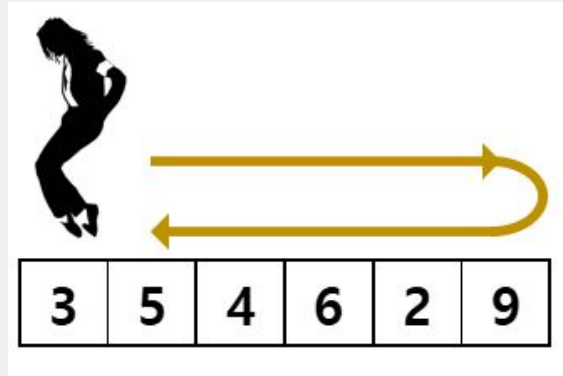


마이클잭슨 무브먼트

문제 3번

마이클잭슨은 **앞으로 갔다가 뒤로가는 백스텝 춤**을 추곤합니다.

이 무브먼트(움직임)를 **재귀를 사용해 출력** 해주세요.



만약,

3 5 4 6 2 9 를 입력 받으면

3 5 4 6 2 9 2 6 4 5 3 이 출력 됩니다.

입력 예제

1 2 3 4 5 6

출력 결과

1 2 3 4 5 6 5 4 3 2 1



두칸씩 점프하기

문제 4번

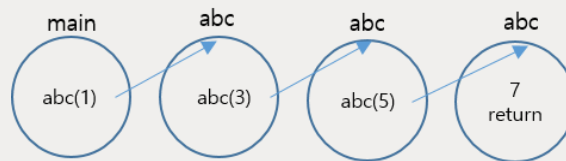
다음과 같이 동작하는 프로그램을 작성해 주세요

한번 재귀호출 될 때 마다, 2씩 증가됩니다.

그리고 3번 재귀 함수 진입 후, 리턴되면서 값을 출력 하면 됩니다.

만약 1을 입력받았다면,

출력 결과 : 7 5 3 1



입력 예제

1

출력 결과

7 5 3 1



다섯글자 순차/역순으로 출력

문제 5번

다섯 글자를 배열에 넣어주세요.

그리고 0번~4번 index까지 출력 하고

4번~0번 index까지 출력하는 프로그램을

재귀호출로 만들어 주세요

입력 예제

abcde

출력 결과

abcde

edcba



문제 6번

a, b 숫자 2개를 입력 받고,
재귀호출을 이용하여

a → b → a 까지 출력 해 주세요.

ex) 3 9

3 4 5 6 7 8 9 8 7 6 5 4 3

입력 예제

3 9

출력 결과

3 4 5 6 7 8 9 8 7 6 5 4 3



문제 7번

3	7	4	1	9	4	6	2
---	---	---	---	---	---	---	---

index 숫자 한개를 입력받으세요

해당 index부터 0번 index까지 출력 한 후

0번 index부터 입력받은 index까지 출력 하는 프로그램을 작성 해 주세요.

재귀호출을 이용하여 문제를 풀어주세요.

ex) 3

1 4 7 3 7 4 1

입력 예제

3

출력 결과

1 4 7 3 7 4 1



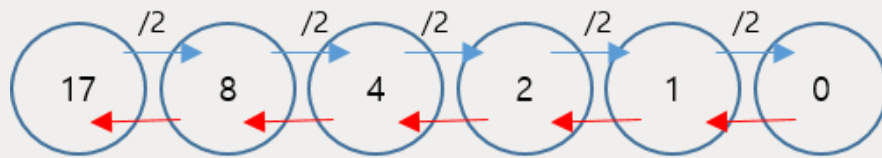
문제 8번

숫자 1개를 입력받고

그 숫자가 0이 될때까지 2로 나누어 주세요

0이 된 이후에는 return하기 시작하여

되돌아 가는 값을 출력 하면 됩니다.



출력 결과 : 1 2 4 8 17

ex) 17

1 2 4 8 17

입력 예제

17

출력 결과

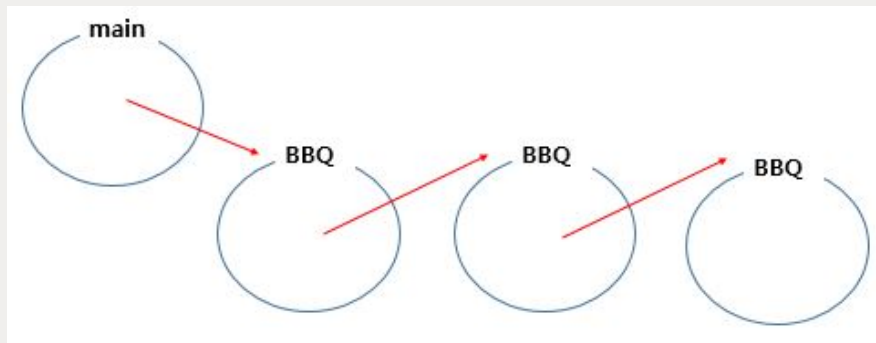
1 2 4 8 17

훈련문제



문제 1번

그림과 같이 동작하는 프로그램을 만들어주세요. (입출력 값 없음)



앞으로 돌진하는 계단

문제 2번

한 문장을 입력 받으세요.(최대 10글자)

그리고 예제와 같이 계단식으로 출력하세요.

중첩 2중 for문을 이용해서 풀어주세요

for문을 돌리는 방향 및 순서를 유의 해 주세요

입력 예제

78ATQP

출력 결과

P

QP

TQP

ATQP

8ATQP

78ATQP



절반 나누기

문제 3번

한 문장을 입력 받으세요.(최대 10글자)

그리고 문장의 길이/2 를 하여 2등분 해주세요.

예를들어 "ABCDEAB" 문장을 2등분하면 $7/2=3$ 이므로

0~2번 index 글자: ABC

3~6번 index 글자: DEAB

이렇게 2등분 할 수 있습니다.

이렇게 나누어진 두 문장이 동일한 문장인지 확인하는 프로그램을 작성 해주세요.

ex1)

입력: ABCABC

출력: 동일한문장

ex2)

입력: ABCDEAB

출력: 다른문장

입력 예제

ABCABC

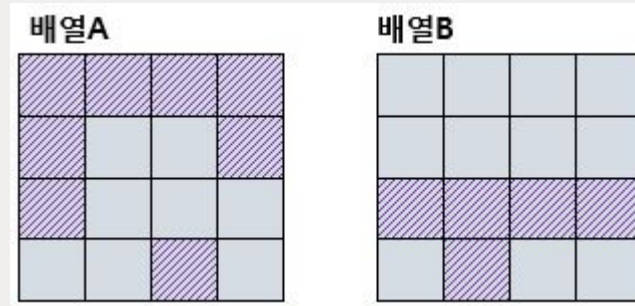
출력 결과

동일한문장



문제 4번

두 비트배열을 입력 받아주세요.



두 비트배열을 겹쳤을때,

색칠한 부분이 겹치면, "걸리다" 출력

겹치는 부분이 없다면, "걸리지않는다" 라고 출력 해주세요.

(색칠된곳은 1로 입력을 받고, 색칠안된곳은 0으로 입력 받으면 됩니다)

입력 예제

1 1 1 1

1 0 0 1

1 0 0 0

0 0 1 0

0 0 0 0

0 0 0 0

1 1 1 1

0 1 0 0

출력 결과

걸리다



문제 5번

문자 1개를 입력받으세요

ASCII코드값 기준 입력 받은 문자에서 부터 -3 문자부터 +3 문자까지 출력하면 됩니다.

만약 G를 입력받았다면 DEF**G**HIJ 를 출력하면 됩니다.

만약 N를 입력받았다면 KLMNOPQ 를 출력하면 됩니다.

그런데 만약 출력해야 하는 문자가 A 이전이라면, Z를 출력하면 되고,

만약 출력해야 하는 문자가 Z 다음 문자라면 A를 출력하면 됩니다.

따라서

만약 Y를 입력받았다면 VWXYZAB 를 출력하면 되고,

만약 B를 입력받았다면 YZABCDE를 출력하면 됩니다.

입력 예제

Y

출력 결과

VWXYZAB



문제 6번

숫자 7개를 배열에 입력 받아주세요.

1층 3번 index까지 출력

2층 4번 index까지 출력

3층 5번 index까지 출력

4층 6번 index까지 출력

ex)

3 5 7 1 4 2 8 을 입력 받았다면 아래와 같이 출력 하면 됩니다.

3 5 7 1

3 5 7 1 4

3 5 7 1 4 2

3 5 7 1 4 2 8

중첩 2중 for문을 활용해서 출력 해주세요.

입력 예제

3 5 7 1 4 2 8

출력 결과

3 5 7 1

3 5 7 1 4

3 5 7 1 4 2

3 5 7 1 4 2 8



문제 7번

한문장을 입력 받으세요.

한문장을 아래와 같이 출력 해주세요.

ex)

[입력]

BBQWORLD

[출력]

B

BB

BBQ

BBQW

BBQWO

BBQWOR

BBQWORL

BBQWORLD

ex)

[입력]

GDPK

[출력]

G

GD

GDP

GDPK

입력 예제

BBQWORLD

출력 결과

B

BB

BBQ

BBQW

BBQWO

BBQWOR

BBQWORL

BBQWORLD

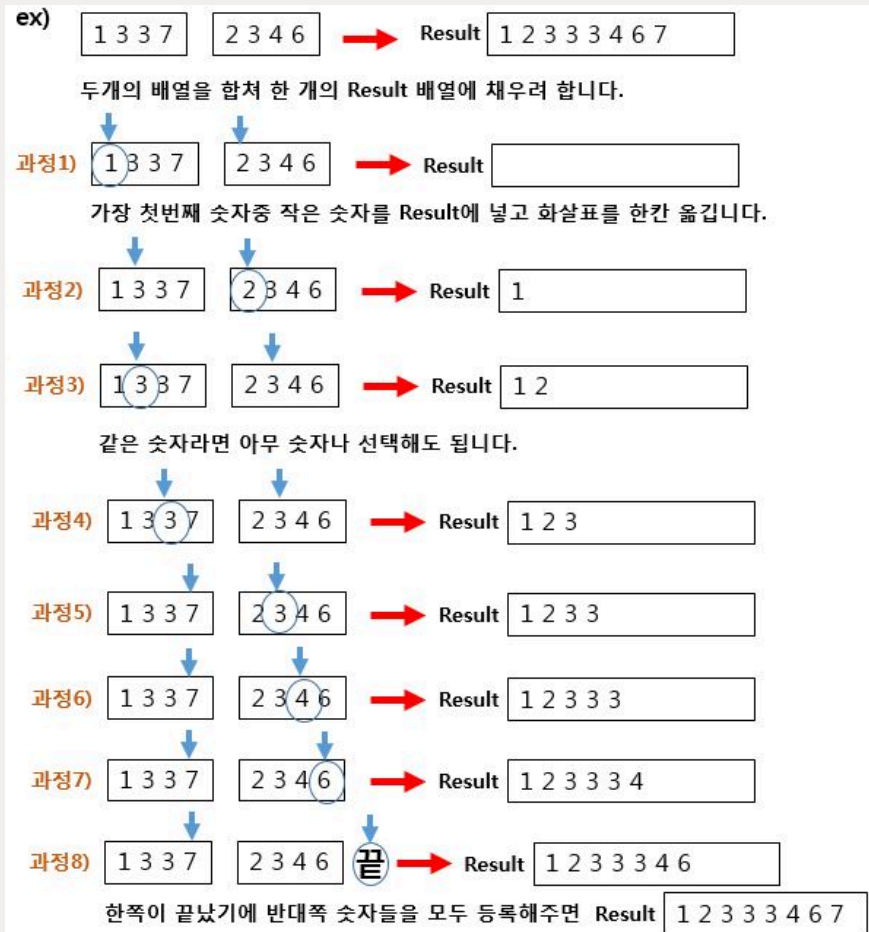


문제 8번

정렬되어 있는 네칸짜리 두 배열에 각각 숫자 4개씩 입력 받습니다.

두 배열에 있는 8개 숫자들을 합쳐 하나의 배열에 정렬된 상태로 넣으려고 합니다.

아래의 알고리즘으로 동작되도록 코딩해주세요.



입력 예제

1 3 3 7

2 3 4 6

출력 결과

1 2 3 3 3 4 6 7



문제 9번

3	5	4	2	5
3	3	3	2	1
3	2	6	7	8
9	1	1	3	2

위 배열을 **하드코딩** 해주세요. **패턴 size**를 입력 받으세요.
만약 **2,2** 를 입력 받았다면

2×2

패턴을 적용 시키면 됩니다.

그리고 패턴을 적용시키면 합을 구할 수 있습니다.

ex) 2,2 size의 패턴을 0,0에 적용시키면 합은 $3+5+3+3=14$ 입니다.

패턴의 size를 입력받고 패턴을 적용시켜 합을 구하여. **max값이 나오는 위치를 출력** 해주세요.

주의 : 입력하실때 y좌표부터 입력받아주세요 ex) 2,3은 y=2, x=3

입력 예제

2 2

출력 결과

(2,3)

객체지향 문제



main()의 함수를 참고하여 Tower 클래스를 작성하라.
높이와 너비는 자유롭게 작성해라.

```
int main()
{
    Tower myTower;
    Tower seoulTower(100);
    cout << "높이는 " << myTower.getHeight() << "미터" << endl;
    cout << "높이는 " << seoulTower .getHeight() << "미터" << endl;
    return 0;
}
```



다음과 같은 Person 클래스가 있다.

Person 클래스와 main() 함수를 작성하여, 3개의 Person 객체를 가지는 배열을 선언하고, 다음과 같이 키보드에서 이름과 전화번호를 입력받아 출력하고 검색하는 프로그램을 완성하라

```
class Person
{
public:
    Person();

    string getName() { return name; }
    string getTel() { return tel; }
    void set(string name, string tel);

private:
    string name;
    string tel;
}
```